Liveness and Register Allocation

CS 1622

Jarrett Billingsley

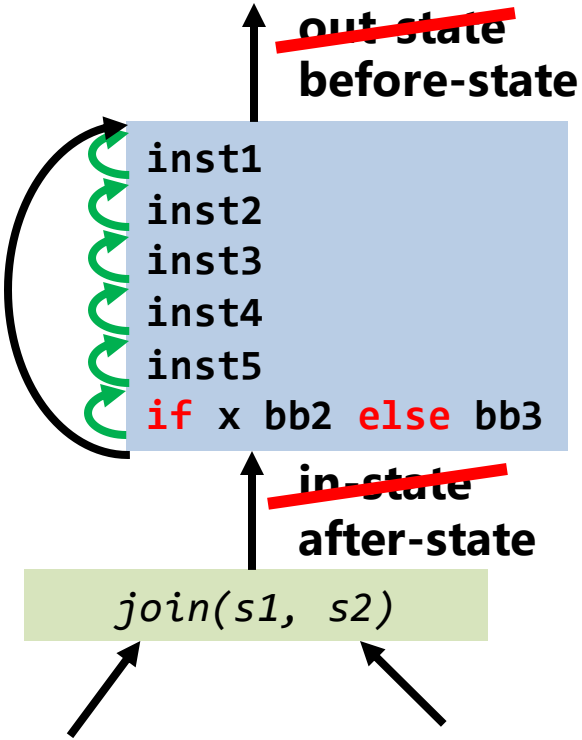
Class Announcements

- uhhhhhh exam Wednesday! woo!
- keep in mind this stuff from today, while interesting, is not likely to be covered *in-depth* on the exam for obvious practical reasons
- also I should acknowledge Stephen Chong of Harvard's CS department as I would not have understood this stuff myself nearly as well without the slides from his compilers course

The liveness algorithm

Backwards Analyses

- it sounds silly, but some analyses make more sense to do *backwards*.



a **backwards analysis** works *exactly the same way,* but in the opposite direction: state flows *from the **successors*** into the join function.

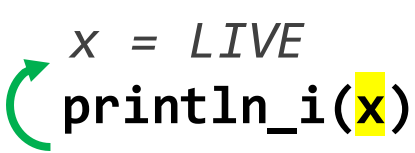
the terms "in-state" and "out-state" take on different meanings, so it can get confusing…

so I'm going to use **before** and **after** instead of **in** and **out.**

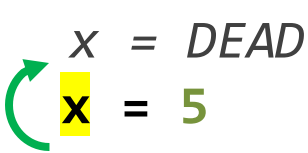
in forward, state goes **before** → **after**;
in backward, state goes **after** → **before**.

States and transfer function

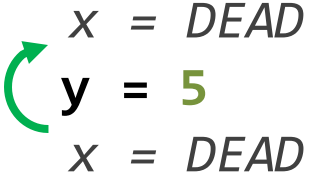
- let's just focus on one variable **x** for now.
- **x** can be in one of two states: **DEAD** or **LIVE**.
  - **DEAD** is also the initial value, so the whole CFG is filled with it.
  - **DEAD** is *also* pulling triple duty as the "unvisited/unknown" value.
- the transfer function works like this:

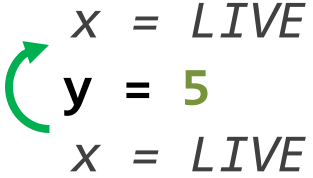


if an instruction **uses x,** then **x** is **LIVE before** it.



else, if an instruction **defs x,** then **x** is **DEAD before** it.





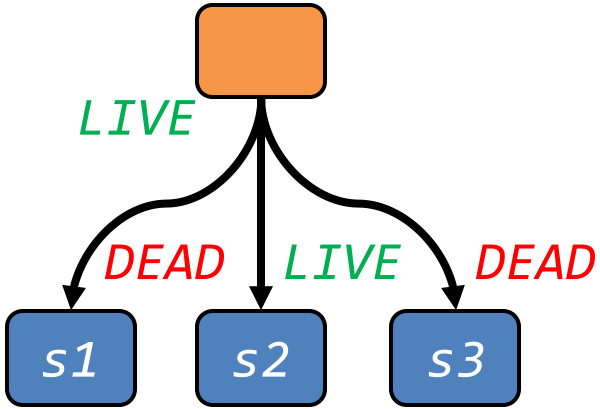
else, before = **after** (just copy the state).

(in **x = x + 1**, the first rule takes precedence, so **x** is **LIVE** before it.)

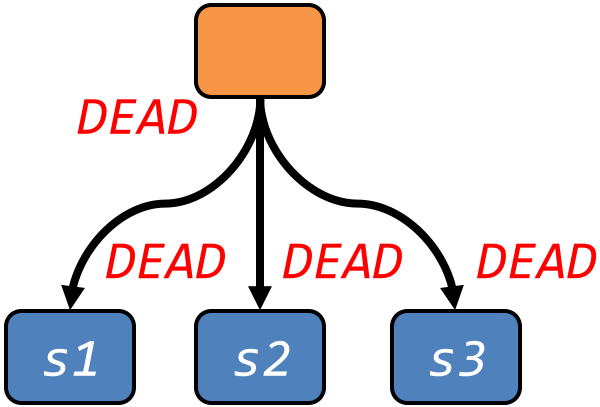
The join function

- the join function is pretty simple.

if **x** is **LIVE** at the beginning of **any** successor, then **x** is **LIVE** at the end of this block.



else, **x** is **DEAD** at the end of this block.

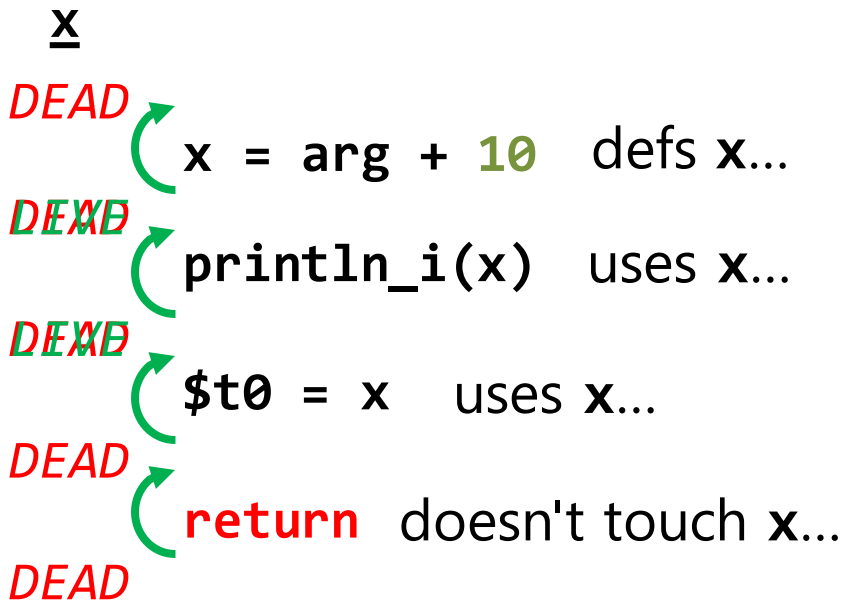


(yeah, in *our* IR, BBs can have at most 2 successors, but this rule works for any number of successors.)

Trying it out (animated)

- let's try it on this simple, one-BB function.

we initialize it by setting **x** to **DEAD** everywhere.



then we run the transfer function on each instruction.

and there we go!

notice how on the last step (the *first* instruction), **x** was marked dead before it, so **we didn't have to change that state.**

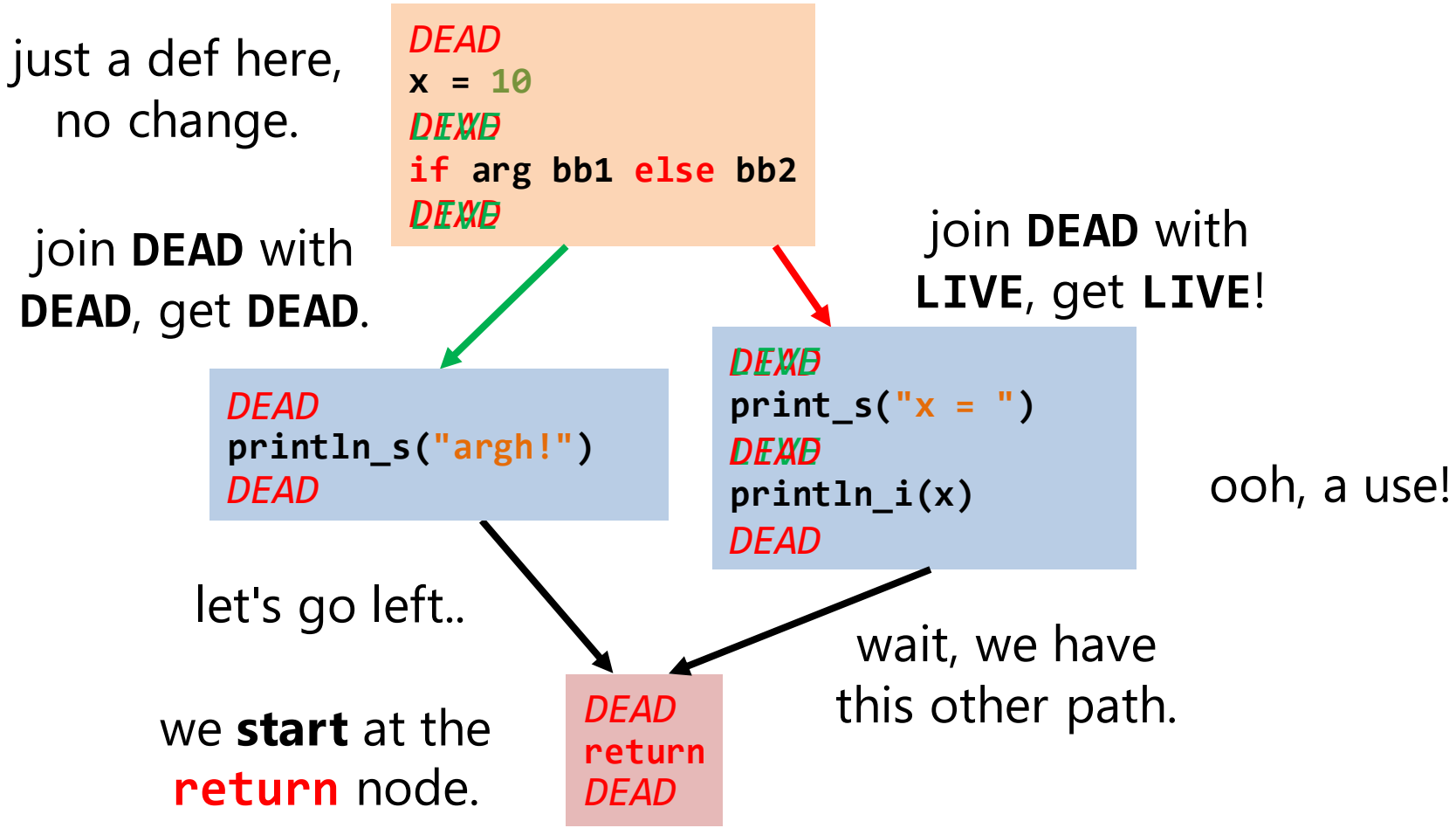
and? why's that matter?

It matters for termination

- we have a finite number of states (2)...
- *and* the states only change **monotonically:**
  - every location **starts** as **`DEAD`**.
  - a location's **`DEAD`** can become **`LIVE`,** but not the other way around.
  - "changing" **`DEAD`** to **`DEAD`** like in that last step is not breaking the rule, because you're not changing anything!
- so we've satisfied the conditions for termination!
  - let's try it on some more complex functions to convince ourselves.

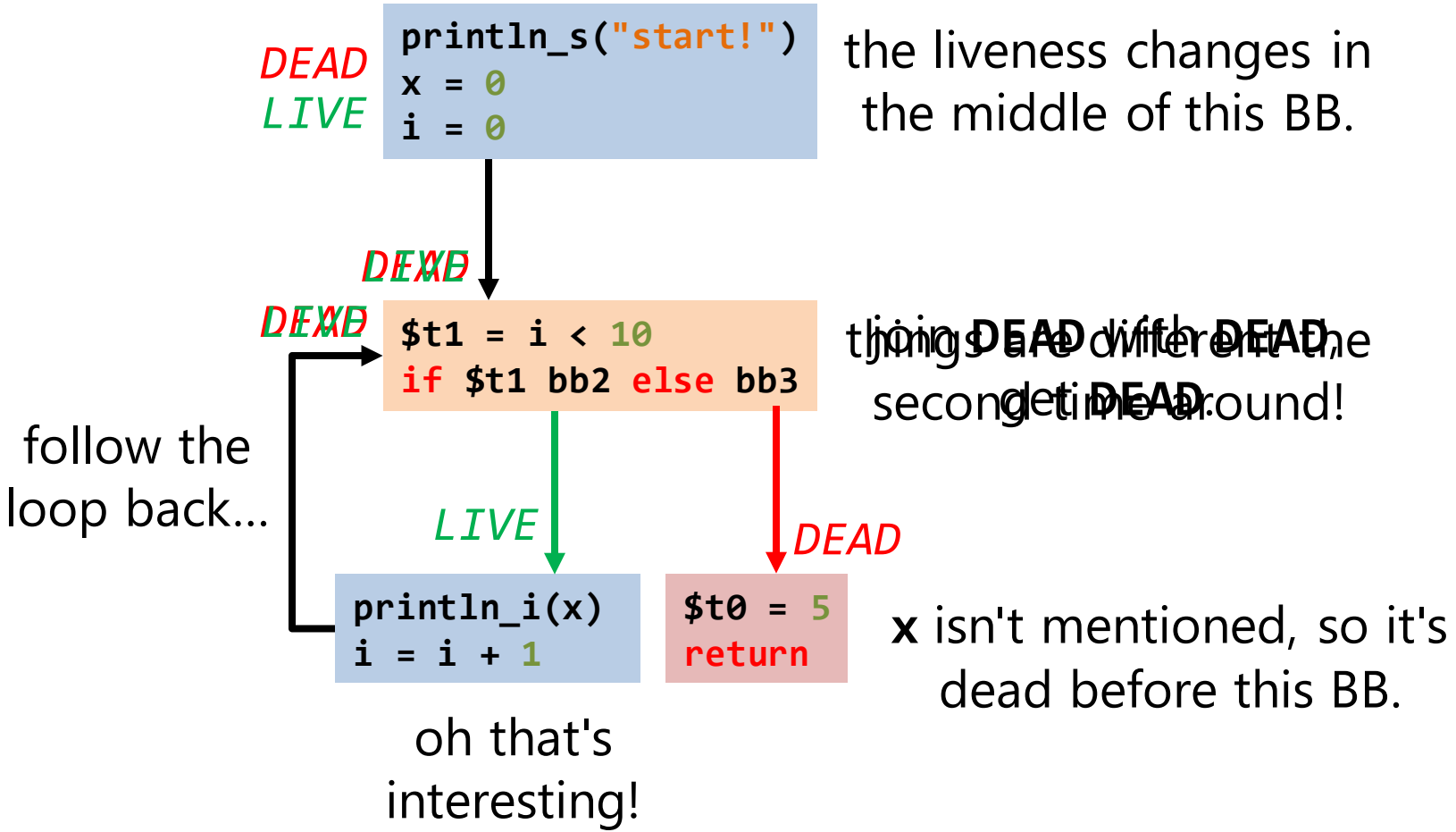
One with a diamond (animated)

- a diamond shape will force us to visit some nodes twice.



One with a loop (animated)

- this should be interesting...



Wait, is that right…?

- these might look wrong at first, but the liveness algorithm is telling you something important about **x**.

```
fn f(a: int) {
    let x = whatever();
    println_i(a);
}
```

x is *always dead??*

yeah, the code **never used x.** maybe the compiler would report a warning/error here.

```
    let x = 10;
    x = 20;
    println_i(x);
```

x is only live for *one* instruction??

the code **never uses the first value** assigned to **x**. maybe a warning/error again!

Some implementation thoughts

- there are only **two states.**
  - so we could store the state of one variable with **one bit.** efficient!
- therefore, to track *n* variables' states, we'd need *n* bits per location.
  - e.g. if we have 3 variables **x**, **y**, **z...**
    - then **`000`** would mean all 3 are dead
    - and **`010`** would mean **y** is live and **x**/**z** are dead.
- this places liveness into a special class of analysis problems called **bit vector problems,** and you can guess why they're called that.
  - bitwise operations are fast, and bits are tiny!
  - think about the join function: if **any** of the successors are live (1)...
    - that means just ORing together all the successors.
- okay. enough about liveness. how do we allocate registers?

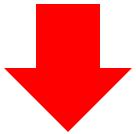
Register Allocation

time check ≤ 27

State of the art circa 1980

- **currently,** our compiler places *all* locals (including args) **in memory.**
  - so, accessing them requires loads and stores.

```
x = y + z;
```



```
lw  s0, -12(fp)
lw  s1, -16(fp)
add s0, s0, s1
sw  s0, -20(fp)
```

this is definitely **correct** (our main goal), but it has serious performance problems.

in, say, 1980, the memory in your computer was **faster than the CPU.**

so loads, stores, and adds all took the **same amount of time.**

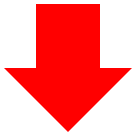
that is absolutely not the case anymore.

in the absolute worst case, a load or store can take around **100 times longer than an add.**

Not all doom and gloom

- due to Reasons You Learn In 1541™, that's just the *worst* case performance. some memory accesses *are* as fast as an add!
- but memory accesses still cause issues for the CPU's pipeline.
- and it's still, like, *more instructions*. cause what if our variables lived in registers instead?

```
x = y + z;
```



```
add s0, s1, s2
```

this is one of the reasons RISC ISAs like MIPS have so many registers to begin with: so you can use them for stuff like this.

more values in registers
means **fewer in memory**
means **fewer loads/stores**
means **higher performance.**

Real ABIs use registers

- as you learned in 447, the real MIPS ABI has four main kinds:
  - **`a`** registers, for **passing arguments** (which we aren't using)
  - **`v`** registers, for **returning values**
  - **`t`** registers, for **temporaries** which do not cross **`jal`**s
  - **`s`** registers, for **temporaries** which do cross **`jal`**s
- our compiler isn't even using **`a`** or **`t`** registers! how wasteful.
- register allocation algorithms allow us to set up constraints to say "hey, this value *must* go in **`a0`**" or similar, so that we can generate code that follows these real ABIs.

Under pressure

- the core idea of register allocation is simple:
  - a function has ***v*** variables, and the CPU has ***r*** registers.
  - if ***v*** ≤ ***r***, it's easy: assign each variable to a register. done!
  - if ***v*** > ***r***, put (***v*** - ***r***) variables in memory, and the rest in registers.
- but this is *too* simplistic, even for simple functions.
  - our IR creates a **lot** of temporary variables.
  - this increases **register pressure:** the number of values we "want" to keep in registers at any given time.
- some optimization passes *also* increase register pressure!
  - e.g. **function inlining** can really blow up the number of locals.
- so, we need something that tells us **which variables are "in use" at any given time,** and assign registers based on that.
  - ...
  - ...oh right, that's liveness :)

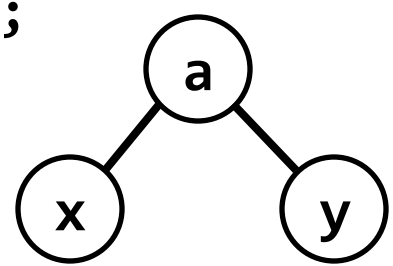
The Register Interference Graph

time check ≤ 40

Oh boy, more graphs! Whee!

- the Register Interference Graph (**RIG**) is the data structure used for register allocation. it is an **undirected graph** with:
  - **one node** for **each local** in the function; and
  - an **edge** between any two locals which are **live at the same time.**
- here is a small function and its RIG.

```
fn riggy(a: int): int {
  let x = a + 10;
  println_i(x);
  let y = "---";
  println_s(y);
  return a;
}
```



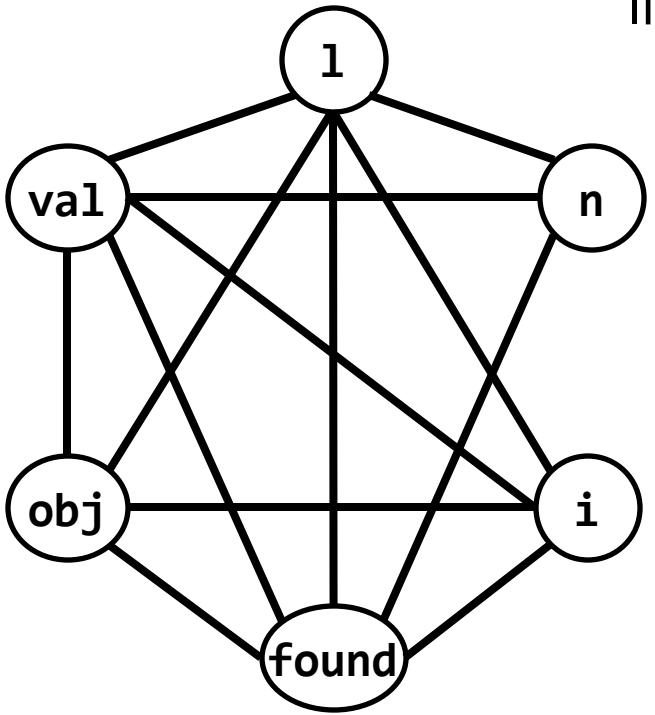
these two edges indicate that **a** and **x** are live at the same time, and so are **a** and **y**, but **x** and **y** are *not* live at the same time.

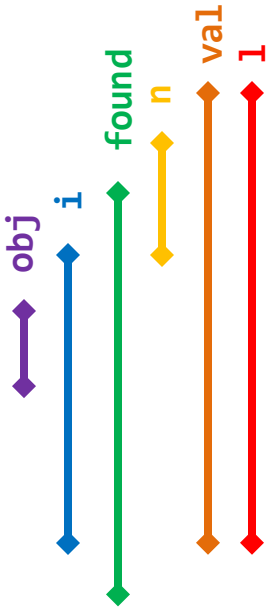
clearly, "live at the same time" is derived directly from the liveness we computed!

Building the RIG (animated)

- after you've computed liveness, it's really straightforward.
- you start with a graph with **no edges.**

at **every point** in the function, if two locals are live at that point, add an edge between them.

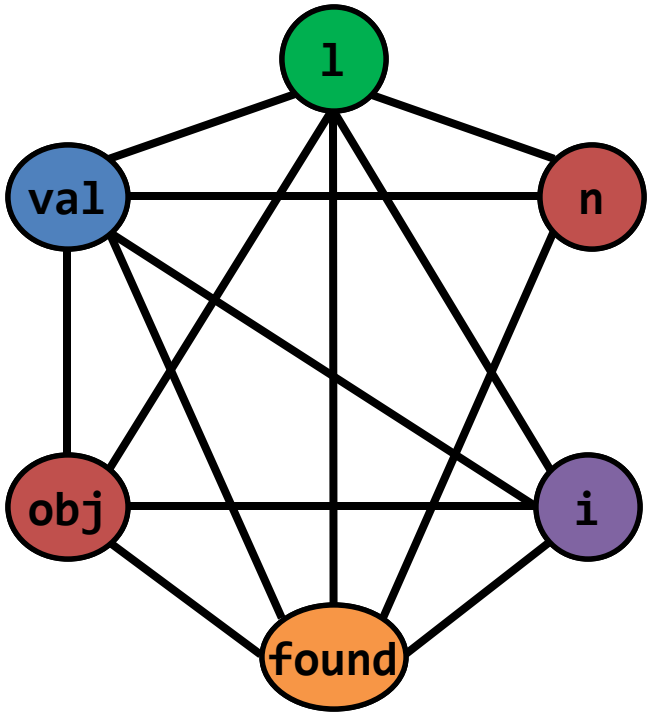





```
fn has(l: List, val: int): bool {
  let n = l.length();
  let found = false;
  for i in 0, n {
    let obj = l.get(i);
    if obj.value() == val {
      found = true;
    }
  }
  return found;
}
```

Ok but what's it mean and *why is it a graph*

- it's the **register interference** graph: an edge between two nodes means that **those two locals cannot go in the same register!**
  - but the *absence* of an edge means they *can* go in the same register.
- as for "why a graph," well, lots of things **reduce to** graph algorithms.



this is **graph coloring:** we assign each node a color such that **no two nodes of the same color** share an edge (touch).

the "color" is just an arbitrary label or set: for our purposes, "color" will mean **the register used to store the variable.**

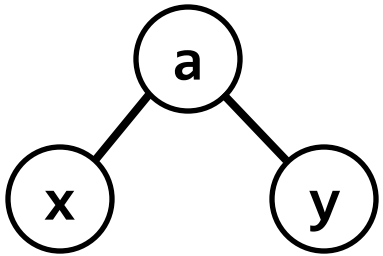
here, **n** and **obj** are the same color, so they can use the same register!

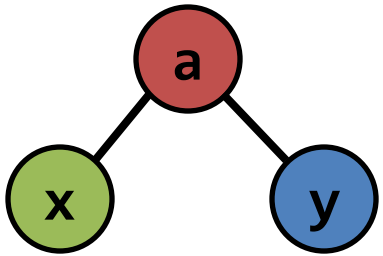
Graph Coloring

time check ≤ 56

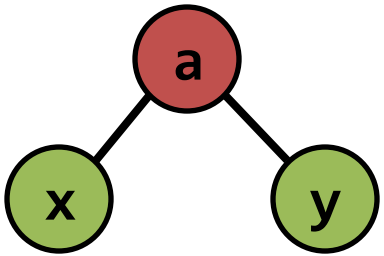
Colorability and chromatic number

- because our "colors" represent the registers on the target CPU…
  - we have a **fixed number of colors to use.**
- we say a graph is *k*-colorable if it can be colored with *k* colors.





we could color this graph with 3 colors…



or just 2. **but 2 is the minimum.**

the **chromatic number** of a graph is the **minimum number of colors needed to color it.**

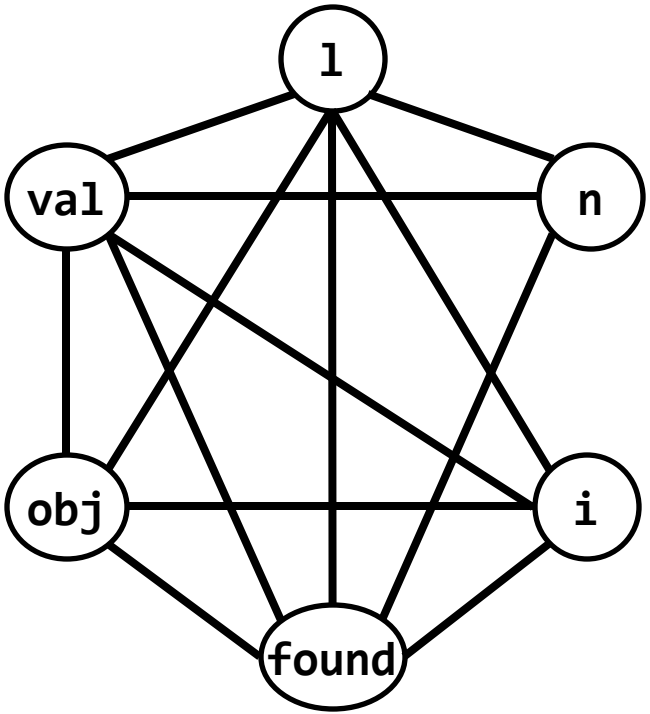
if we have *r* registers, we'd really like **the chromatic number of the RIG to be ≤ r!**

You didn't think it would be THAT easy, did you

- unfortunately, **determining k-colorability is NP-complete.**
  - (and determining the chromatic number is NP-hard!)
- so, what do we do?
  - sometimes, an **imperfect solution** is good enough.
- we're going to use a **heuristic** to figure out a **coloring order** that is likely to lead to a successful coloring.
  - and if that fails, we can **tweak the graph a bit** and try it again.
- this leads to an iterative solution that is **near-linear** for most common cases.
  - yeah we might not find the *perfect* register allocation, but perfection is the enemy of good.

The coloring heuristic (animated)

- let's say ***r* = 5** (where *r* = number of CPU registers).
- if the RIG has a node with < *r* neighbors, and you remove that node...
  - **and** the **resulting graph** is *r*-colorable...
  - then the **original graph is *r*-colorable.**



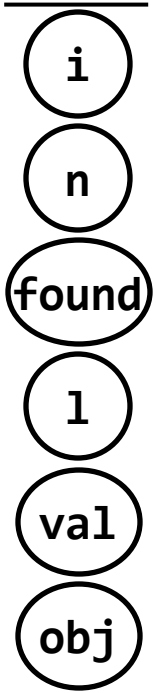
Stack

we'll repeatedly remove nodes with < 5 neighbors (edges), and **put them in this stack** as we do so.

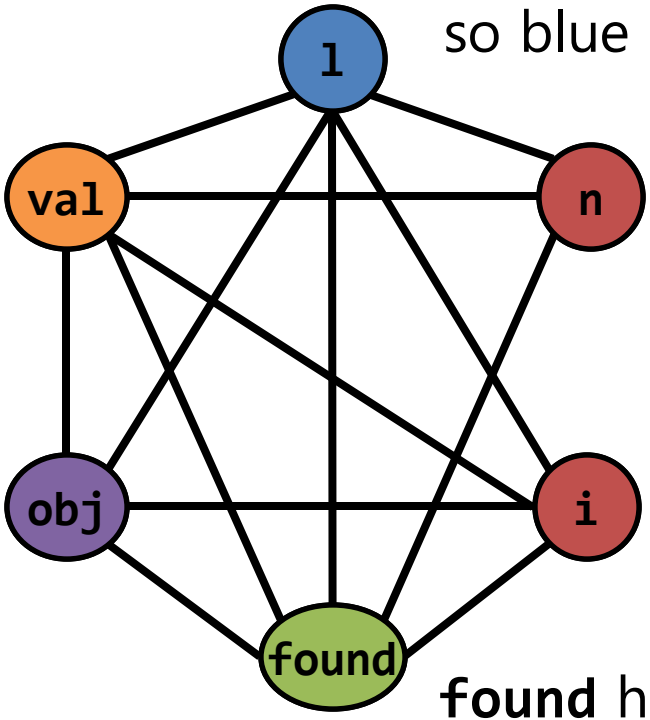
Pop'n'color (animated)

- now we **pop** the nodes off the stack, and as we do so, **color them** according to what their neighbors are.

Stack



`l` has red and green neighbors, so blue it is.



`n` has no colored neighbors, so let's make it red too.

`val` needs to be a fourth color...

and **`obj`** needs to be a fifth.

`i` has no colored neighbors, so let's make it red.

`found` has two red neighbors, so let's make it green.

This works??

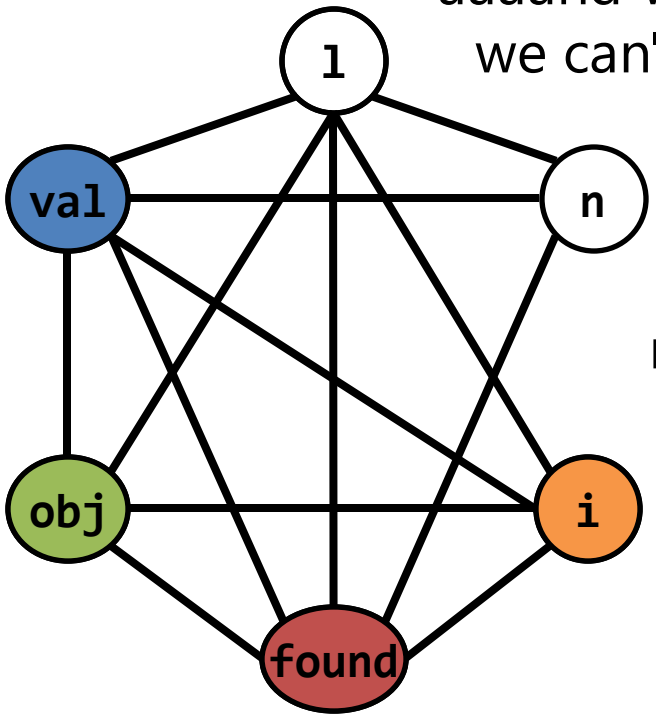
- it's a little mind-blowing, but it does work, because that heuristic actually goes **both ways:** it's an "if and only if."
  - because we were able to 5-color each sub-graph of the RIG as we built it back up by popping…
  - then the whole RIG was 5-colorable.
- BUT: it doesn't *always* work. if the RIG is not actually *r*-colorable, we will run into one of two things:
  - no nodes with < r neighbors on the removal phase; or:
  - impossible-to-color nodes in the popping phase!

Failure to color

- let's see what happens on the same RIG if $r = 4$ instead.
  - we have to remove nodes with < 4 neighbors now.

Stack

found
obj
val
i
l
n



aaaand we're stuck.
we can't color **l**!

uh oh. **everyone has ≥ 4 neighbors.** let's just remove one and move on anyway.

Spilling

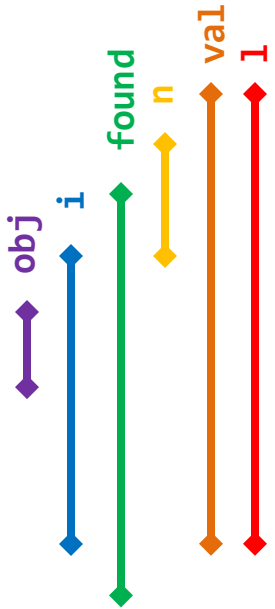
time check ≤ 85

So what do we do??

- a failure to color the RIG means that **there are more values being used at one time than will fit into the CPU's registers.**
  - it's a setback, but it's not *catastrophic,* right? we *can* use memory.
- so in this case we turn to **spilling:** picking one or more locals to live *on the stack* instead of in registers.
- ideally, we'd like to pick some local that **isn't used very often** to be placed into memory instead of a register... but **which one?**
  - if we pick poorly, we could slow down our program a lot by causing a bunch of excess loads and stores...

Stop interfering with me!

- picking which local to spill is, again, something we can't do perfectly.
- so we can again use heuristics to pick a likely candidate. maybe some variable that **interferes with a lot of others** but isn't used much…




```
fn has(l: List, val: int): bool {
  let n = l.length();
  let found = false;
  for i in 0, n {
    let obj = l.get(i);
    if obj.value() == val {
      found = true;
    }
  }
  return found;
}
```

locals with **long live ranges** tend to interfere a lot.

we also have to consider things like loops – how many accesses to the local will happen because of the loop? can we estimate that?

found seems like a good spilling candidate here.

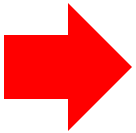
So how does it work?

- if we **get stuck** when removing nodes from the RIG…
  - we pick a local to *maybe* spill. **but we don't spill it yet.**
  - we push that node on the stack, **along with a note** that says, "maybe spill this?"
- then, when we pop-and-color…
  - if we get to a maybe-spill entry on the stack...
  - AND that node cannot be colored…
  - then we know that we have to spill it, and we **rewrite the code!**
- why do we wait to rewrite until the pop-and-color phase?
  - because we're just using **heuristics** here. it's entirely possible that the graph *is* colorable when we remove a node with ≥ *r* neighbors! so, we have to be conservative.

How the code is rewritten

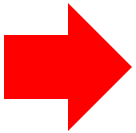
- in our IR, we might add new instructions to represent loads/stores.
- then, each use and def of the spilled variable is replaced with loads and stores to the stack.

```
found = false
```



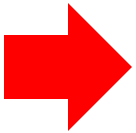
```
found_1 = false
Store(fp-12, found_1)
```

```
found = true
```



```
found_2 = true
Store(fp-12, found_2)
```

```
$t0 = found
```

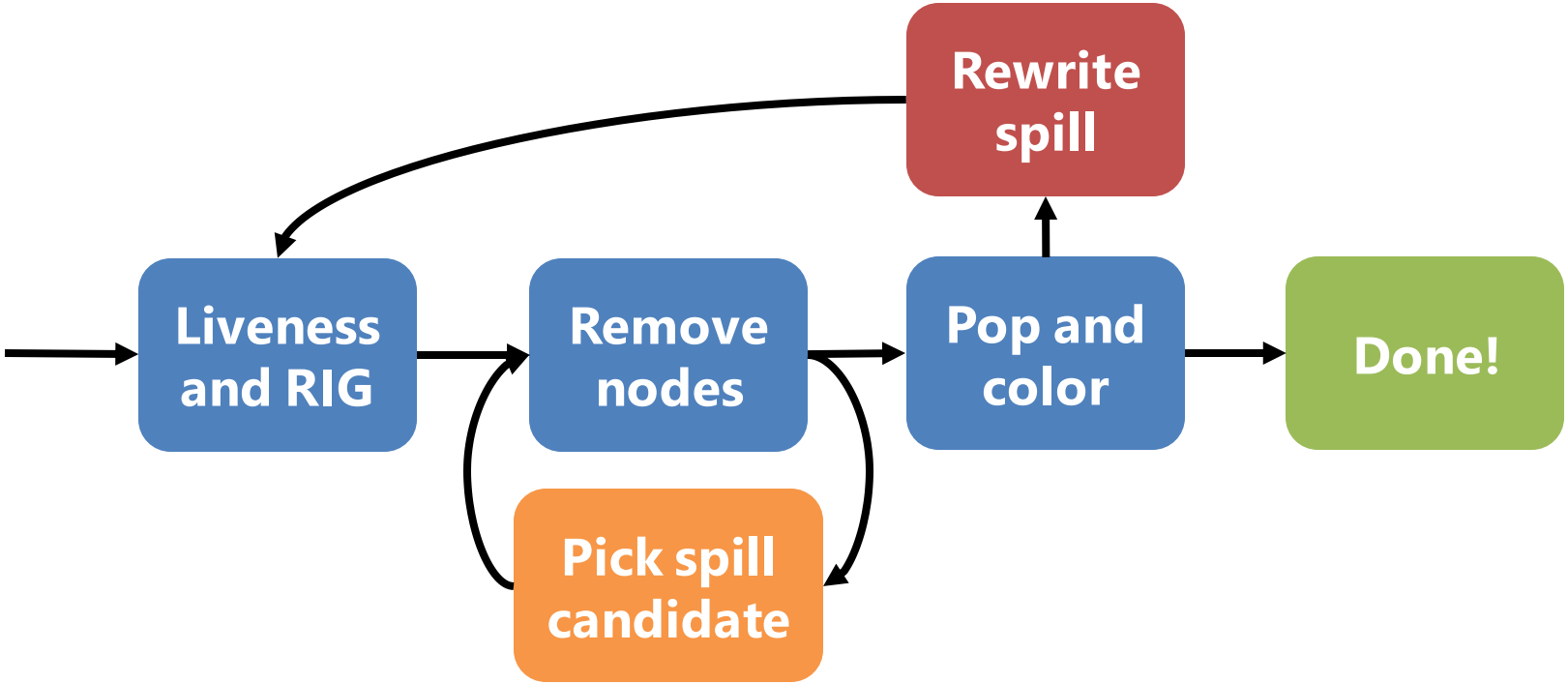


```
found_3 = Load(fp-12)
$t0 = found_3
```

we also **rename** the spilled local at each use location with a unique name. each of these new locals has a very short live range!

Then what?

- then… we **start all over again!** compute liveness, build the RIG…
- this is the iterative register allocation algorithm in a nutshell:



And it keeps going…

- this is just the basic shape of the algorithm.
- there are several more features, such as:
  - **coalescing** pairs of nodes that are likely to end up in the same register anyway
    - e.g. in **a = b** where their liveness doesn't overlap
  - **pre-coloring** some nodes to indicate they must be in certain registers. this is used for:
    - argument and return value registers (MIPS "a" and "v" registers)
    - saved temporary registers (MIPS "s" registers)
  - and more…